

Matplotlib

is the foundational plotting library in Python. Its primary interface is pyplot, which provides a MATLAB-like environment for creating static, animated, and interactive visualizations . It gives you complete control over every element of a figure, from axis labels to line styles.

1. The Core Concepts: Figures & Axes

- **Figure:** The entire window or page that everything is drawn on (the "canvas").
- **Axes:** The individual plot area where data is plotted (the "chart"). A figure can contain multiple Axes objects (subplots).

2. Import Convention

```
python
import matplotlib.pyplot as plt
import numpy as np
```

3. Creating a Basic Plot

Function	Description	Example
plt.plot(x, y)	Create a line plot (connects points) .	plt.plot(x, y)
plt.scatter(x, y)	Create a scatter plot (individual points) .	plt.scatter(x, y)
plt.bar(x, height)	Create a vertical bar chart .	plt.bar(categories, values)
plt.barh(y, width)	Create a horizontal bar chart .	plt.barh(categories, values)
plt.hist(data)	Create a histogram (frequency distribution) .	plt.hist(data, bins=20)
plt.boxplot(data)	Create a box-and-whisker plot .	plt.boxplot([data1, data2])
plt.pie(sizes)	Create a pie chart .	plt.pie(sizes, labels=labels)

Function	Description	Example
<code>plt.imshow(array)</code>	Display an image or heatmap .	<code>plt.imshow(matrix, cmap='viridis')</code>
<code>plt.contour(X, Y, Z)</code>	Create a contour plot .	<code>plt.contour(X, Y, Z)</code>
<code>plt.subplot()</code> / <code>plt.subplots()</code>	Create multiple plots in a single figure .	<code>fig, axes = plt.subplots(2, 2)</code>

4. Customizing Your Plot

Function	Description	Example
<code>plt.title('Title')</code>	Add a title to the plot .	<code>plt.title('My Scatter Plot')</code>
<code>plt.xlabel('Label')</code>	Add a label to the x-axis .	<code>plt.xlabel('X values (units)')</code>
<code>plt.ylabel('Label')</code>	Add a label to the y-axis .	<code>plt.ylabel('Y values (units)')</code>
<code>plt.xlim(min, max)</code>	Set the limits of the x-axis .	<code>plt.xlim(0, 10)</code>
<code>plt.ylim(min, max)</code>	Set the limits of the y-axis .	<code>plt.ylim(-5, 5)</code>
<code>plt.legend()</code>	Show a legend. Requires a label in your plot commands .	<code>plt.plot(x, y, label='Line 1')</code> and then <code>plt.legend()</code>
<code>plt.grid(True)</code>	Add a grid to the plot .	<code>plt.grid(True, linestyle='--')</code>
<code>plt.xticks(rotation=45)</code>	Rotate x-axis tick labels (very useful for dates/categories) .	<code>plt.xticks(rotation=90)</code>
<code>plt.axhline(y)</code>	Add a horizontal line at a specific y-value .	<code>plt.axhline(y=0.5, color='red')</code>
<code>plt.axvline(x)</code>	Add a vertical line at a specific x-value .	<code>plt.axvline(x=100, color='k')</code>

5. Controlling Plot Aesthetics (Line Styles, Colors, Markers)

When you call `plt.plot()`, you can format the line in a single string: `fmt = '[color][marker][line]'` .

Parameter	Options	Example
Color	'b', 'g', 'r', 'c', 'm', 'y', 'k' (black), 'w' (white) .	plt.plot(x, y, 'r')
Line Style	'-' (solid), '--' (dashed), '-.' (dash-dot), ':' (dotted) .	plt.plot(x, y, '--')
Marker	'o' (circle), 's' (square), '^' (triangle), 'x' (cross), '.' (point), '*' (star) .	plt.plot(x, y, 'o')

Combined Example: plt.plot(x, y, 'ro--') plots red circles connected by a dashed line.

Advanced Formatting (Explicit Parameters):

python

```
plt.plot(x, y, color='orange', linestyle='--', linewidth=2, marker='s', markersize=8, alpha=0.7)
```

6. Working with Multiple Plots (Subplots)

Method 1: Using plt.subplot() (Rows, Columns, Index)

python

```
plt.subplot(1, 2, 1) # 1 row, 2 cols, plot 1
```

```
plt.plot(x, np.sin(x))
```

```
plt.title('Sine')
```

```
plt.subplot(1, 2, 2) # 1 row, 2 cols, plot 2
```

```
plt.plot(x, np.cos(x))
```

```
plt.title('Cosine')
```

Method 2: Using plt.subplots() (Recommended for more control)

python

```
fig, axes = plt.subplots(2, 2, figsize=(10, 6)) # Creates 2x2 grid
```

```
axes[0, 0].plot(x, y1)
```

```
axes[0, 1].scatter(x, y2)
```

```
axes[1, 0].bar(categories, values)
```

```
axes[1, 1].hist(data)
```

```
# Adjust layout to prevent overlapping
```

`plt.tight_layout()`

7. Common Customizations & Styles

Function	Description
<code>plt.figure(figsize=(10, 6))</code>	Change the size of the figure (in inches) .
<code>plt.tight_layout()</code>	Automatically adjust spacing between subplots .
<code>plt.style.use('ggplot')</code>	Use a pre-defined style (e.g., 'ggplot', 'seaborn', 'dark_background') .
<code>plt.savefig('plot.png')</code>	Save the current figure to a file (dpi=300 for high-res) .
<code>plt.colorbar()</code>	Add a colorbar to a heatmap/contour plot .
<code>plt.text(x, y, 'text')</code>	Add arbitrary text to the plot .

8. Saving & Displaying

Function	Description
<code>plt.show()</code>	Display the figure on screen .
<code>plt.savefig('filename.png', dpi=300, bbox_inches='tight')</code>	Save to file. Use <code>bbox_inches='tight'</code> to avoid clipping axes labels .

9. Quick Reference: Plot Types in Action

python

--- LINE PLOT ---

```
plt.plot(x, y, label='Data', color='blue', linewidth=2)
```

```
plt.legend()
```

```
plt.show()
```

--- SCATTER PLOT ---

```
plt.scatter(x, y, c=z, cmap='viridis', s=50, alpha=0.5)

plt.colorbar()

plt.show()
```

```
# --- BAR CHART ---

categories = ['A', 'B', 'C']

values = [10, 25, 15]

plt.bar(categories, values, color=['red', 'green', 'blue'])

plt.show()
```

```
# --- HISTOGRAM ---

data = np.random.randn(1000)

plt.hist(data, bins=30, edgecolor='black', alpha=0.7)

plt.show()
```

```
# --- SUBPLOTS ---

fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(10, 8))

ax1.plot(x, np.sin(x))

ax2.scatter(x, np.cos(x))

ax3.hist(np.random.randn(500), bins=20)

ax4.bar(['A', 'B', 'C'], [5, 15, 10])

plt.tight_layout()

plt.show()
```

10. Pro Tips

- **Using** `plt.clf()`: Clears the current figure. Useful in loops to avoid overlapping plots .
- `fig, ax = plt.subplots()`: This is the object-oriented way. Use `ax.plot()` instead of `plt.plot()` for finer-grained control.
- **Check available styles**: `print(plt.style.available)` to see all available style sheets.
- **Data Leakage**: Unlike Scikit-learn, Matplotlib is purely for visualization. It doesn't "remember" data, so you need to keep track of variables yourself.